DAY 6 – PYTHON FILE HANDLING FOR AWS DATA ENGINEERING

Today we will learn:

1. What is File Handling?
2. Why File Handling is important in Data Engineering
3. open() function
4. File opening modes
5. Reading a file using read()
6. Reading using readline()
7. Reading using readlines()
8. Using with open()
9. Writing to a file
10. Appending to a file
11. Difference between w and a
12. Checking whether a file exists
13. Working with file paths
14. Reading CSV files
15. Writing CSV files
16. CSV DictReader
17. CSV DictWriter
18. Processing CSV records
19. Validating CSV data
20. Handling missing files
21. File handling with functions
22. Data Engineering file processing
23. Practice programs

1. What is File Handling?

File handling means creating, reading, writing, and updating files using Python.

For example, in Data Engineering, we receive files such as:

customers.csv

orders.csv

products.csv

sales.txt

pipeline.log

Python can read these files, validate their contents, process the records, and write results into another file.

2. Why is File Handling important?

A simple Data Engineering flow looks like this:

Source CSV File

↓

Python reads the file

↓

Validate records

↓

Transform data

↓

Write processed file

↓

Upload to S3

File handling is one of the basic skills required before working with S3, AWS Glue, and other data processing services.

3. Creating a text file

```
file = open("customers.txt", "w")
file.write("Welcome to AWS Data Engineering")
file.close()
```

This creates a file named customers.txt in the current working directory and writes text into it.

Explanation: open() opens the file, "w" selects write mode, write() writes the text, and close() closes the file.

4. Reading a file using read()

```
file = open("customers.txt", "r")
data = file.read()
print(data)
file.close()
```

Output

Welcome to AWS Data Engineering

Here, "r" means read mode. The read() method reads the entire file content.

5. File opening modes

Mode	Meaning
r	Read an existing file
w	Write, replacing existing content
a	Append content
x	Create a new file; fail if it exists
rb	Read binary data
wb	Write binary data

6. Using with open()

This is the recommended way to handle files in most Python programs.

```
with open("customers.txt", "r") as file:
    data = file.read()

print(data)
```

Output

Welcome to AWS Data Engineering

The advantage is that Python automatically closes the file when the with block finishes, including when an exception occurs.

7. Writing multiple lines

```
with open("pipeline.txt", "w") as file:
    file.write("Pipeline Started\n")
    file.write("Reading Customers\n")
    file.write("Pipeline Completed\n")
```

The \n character creates a new line.

The file will contain:

Pipeline Started

Reading Customers

Pipeline Completed

8. Appending to a file

```
with open("pipeline.txt", "a") as file:
    file.write("New Pipeline Execution\n")
```

Append mode adds content to the end without removing the existing content.

9. Reading line by line

```
with open("pipeline.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Output

Pipeline Started

Reading Customers

Pipeline Completed

New Pipeline Execution

The strip() method removes the newline from each line before printing.

10. Using readline()

```
with open("pipeline.txt", "r") as file:
    first_line = file.readline()

print(first_line.strip())
```

Output

Pipeline Started

readline() reads one line at a time.

11. Using readlines()

```
with open("pipeline.txt", "r") as file:
    lines = file.readlines()

print(lines)
```

Output

['Pipeline Started\n', 'Reading Customers\n', 'Pipeline Completed\n', 'New Pipeline Execution\n']

readlines() returns a list of lines. For very large files, reading one line at a time is usually more memory-efficient.

12. Checking whether a file exists

```
from pathlib import Path

file_path = Path("customers.txt")

if file_path.is_file():
    print("File exists")
else:
    print("File does not exist")
```

Output, assuming the file was created earlier:

File exists

This helps prevent errors when an expected source file is missing.

13. Working with file paths

```
from pathlib import Path

file_path = Path("data") / "customers.csv"

print(file_path)
print(file_path.name)
print(file_path.suffix)
```

Output on Amazon Linux

data/customers.csv

customers.csv

.csv

Path helps us work with file names, directories, and extensions.

14. Creating a CSV file

```
import csv

with open("customers.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["id", "name", "city"])
    writer.writerow([101, "Arun", "Chennai"])
    writer.writerow([102, "Priya", "Bangalore"])
    writer.writerow([103, "Kumar", "Hyderabad"])
```

The generated CSV contains:

id,name,city

101,Arun,Chennai

102,Priya,Bangalore

103,Kumar,Hyderabad

csv.writer() writes CSV rows correctly, including handling commas and quoted values.

15. Reading CSV data

```
import csv

with open("customers.csv", "r", newline="") as file:
    reader = csv.reader(file)

    for row in reader:
        print(row)
```

Output

['id', 'name', 'city']

['101', 'Arun', 'Chennai']

['102', 'Priya', 'Bangalore']

['103', 'Kumar', 'Hyderabad']

Notice that CSV values are read as strings.

16. Using DictReader

```
import csv

with open("customers.csv", "r", newline="") as file:
    reader = csv.DictReader(file)

    for customer in reader:
        print(customer["name"], customer["city"])
```

Output

Arun Chennai

Priya Bangalore

Kumar Hyderabad

DictReader uses the first CSV row as dictionary keys.

17. Using DictWriter

```
import csv

customers = [
    {"id": 101, "name": "Arun"},
    {"id": 102, "name": "Priya"}
]

with open("output.csv", "w", newline="") as file:
    writer = csv.DictWriter(file, fieldnames=["id", "name"])
    writer.writeheader()
    writer.writerows(customers)
```

The generated output.csv contains:

id,name

101,Arun

102,Priya

18. Counting CSV records

```
import csv

record_count = 0

with open("customers.csv", "r", newline="") as file:
    reader = csv.DictReader(file)

    for row in reader:
        record_count += 1

print("Total Records:", record_count)
```

Output

Total Records: 3

The CSV header is not counted because DictReader uses it as field names.

19. Validating CSV records

```
import csv

with open("customers.csv", "r", newline="") as file:
    reader = csv.DictReader(file)

    for customer in reader:
        if customer["name"].strip():
            print("Valid:", customer["name"])
        else:
            print("Invalid customer name")
```

Output

Valid: Arun

Valid: Priya

Valid: Kumar

This checks that the name is not empty or only whitespace.

20. Handling missing files

```
try:
    with open("missing.csv", "r") as file:
        data = file.read()
except FileNotFoundError:
    print("Source file not found")
```

Output, if missing.csv does not exist:

Source file not found

This prevents an unhandled missing-file error from immediately terminating the program.

21. File handling using functions

```
def read_file(file_name):
    with open(file_name, "r") as file:
        return file.read()

data = read_file("customers.txt")
print(data)
```

Output

Welcome to AWS Data Engineering

The function can be reused with different file names.

22. Data Engineering file processing

```
import csv

def process_customer_file(source, target):
    valid_customers = []

    with open(source, "r", newline="") as file:
        reader = csv.DictReader(file)

        for customer in reader:
            if customer["name"].strip():
                valid_customers.append(customer)

    with open(target, "w", newline="") as file:
        writer = csv.DictWriter(
            file,
            fieldnames=["id", "name", "city"]
        )
        writer.writeheader()
        writer.writerows(valid_customers)

    print("Processed Records:", len(valid_customers))

process_customer_file("customers.csv", "valid_customers.csv")
```

Output

Processed Records: 3

This program reads source records, validates customer names, and writes the valid records into a separate CSV file.

23. Day 6 practice programs

Practice these independently:

1. Create employees.txt and write five employee names.
2. Read the file line by line and display each employee.
3. Append two more employees without deleting the original names.
4. Create a CSV with employee_id, name, and salary.
5. Read the CSV and print employees earning more than 50,000.
6. Create a function to count CSV records.
7. Handle a missing source file using try-except.